

Reviewer Pack — Moment-Matrix Rank Deficit in Max-CUT Approximation

Entry 1c394bcaff4a · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-08 13:36:15 UTC

Moment-Matrix Rank Deficit in Max-CUT Approximation

Entry ID: 1c394bcaff4a **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-08 13:36:15 UTC

1. Conjecture

Field A (mathematical branch): REAL_ALGEBRAIC_GEOMETRY **Field B** (complexity object): SOS_DEGREE

Statement:

For any graph G with maximum cut $\alpha(G)$, any degree- d Sum-of-Squares (SOS) relaxation achieving approximation ratio $0.879 + \varepsilon$ must have a moment matrix M_d with rank $\geq \Omega(\sqrt{n})$. This rank threshold is preserved under polynomial reductions between Max-CUT instances.

Rationale (proposer’s reasoning):

The moment matrix rank captures the ‘complexity’ of the SOS relaxation. By linking it to the graph’s structure, we isolate a geometric obstruction to improving the Goemans-Williamson bound. This connects spectral concentration in random matrices (real algebraic geometry) to SOS hierarchy limitations.

Taxonomy category: SOS_DEGREE (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): d3b7d3b2cbbb6867

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.95	SAFE	UNCERTAIN
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	0.95	UNCERTAIN	HITS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=124, elapsed=240.1s

5.1 Generated Python source

```
import random
import math
from itertools import combinations

def generate_random_3_regular_graph(n):
    if n % 2 != 0:
        raise ValueError("n must be even for a 3-regular graph")
    edges = set()
    nodes = list(range(n))
    while len(edges) < n * 3 // 2:
        u, v = random.sample(nodes, 2)
        if (u, v) not in edges and (v, u) not in edges:
            edges.add((u, v))
            for w in nodes:
                if (w, u) in edges or (u, w) in edges:
                    continue
                if (w, v) in edges or (v, w) in edges:
                    continue
            edges.add((u, w))
            edges.add((v, w))
    return edges

def max_cut_value(graph):
    n = len(graph)
    best_cut = 0
    for partition in combinations(range(n), n // 2):
        cut_size = sum(1 for u, v in graph if (u in partition and v not in partition) or (v in partition and u not in partition))
        best_cut = max(best_cut, cut_size)
    return best_cut

def degree_d_moment_matrix(graph, d):
    n = len(graph)
    M = [[0] * (n + 1) for _ in range(n + 1)]
    for i in range(n):
        M[i][i] = 2
    for u, v in graph:
        M[u][v] += 1
        M[v][u] += 1
    return M
```

```

def rank(matrix):
    n = len(matrix)
    augmented_matrix = [row + [0] * (n - len(row)) + [i] for i, row in enumerate(matrix)]
    for i in range(n):
        if augmented_matrix[i][i] == 0:
            for j in range(i + 1, n):
                if augmented_matrix[j][i] != 0:
                    augmented_matrix[i], augmented_matrix[j] = augmented_matrix[j], augmented_matrix[i]
                    break
            else:
                return i
        pivot = augmented_matrix[i][i]
        for j in range(n + 1):
            augmented_matrix[i][j] /= pivot
        for j in range(n):
            if j != i and augmented_matrix[j][i] != 0:
                factor = augmented_matrix[j][i]
                for k in range(n + 1):
                    augmented_matrix[j][k] -= factor * augmented_matrix[i][k]
    return n - sum(1 for row in augmented_matrix if all(val == 0 for val in row[:n]))

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = 40
    d = 2
    epsilon = 0.001
    alpha_G = max_cut_value(generate_random_3_regular_graph(n))
    M_d = degree_d_moment_matrix(generate_random_3_regular_graph(n), d)
    rank_M_d = rank(M_d)
    metric_name = "Rank of Moment Matrix"
    metric_value = rank_M_d
    instances_tested = 1
    conjecture_holds = rank_M_d >= math.sqrt(n) * (1 - epsilon)
    counterexample = "" if conjecture_holds else f"Rank {rank_M_d} <  $\Omega(\sqrt{n})$ "
    return {
        "metric_name": metric_name,
        "metric_value": metric_value,
        "instances_tested": instances_tested,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(x) for x in sys.argv[1:]] or [2**i - 1 for i in range(5, 30)]
    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(res["metric_value"] for res in results) / len(results)
    std_value = math.sqrt(sum((res["metric_value"] - mean_value)**2 for res in results) / len(results))
    support_fraction = sum(1 for res in results if res["conjecture_holds"]) / len(results)

    if all(res["conjecture_holds"] for res in results):

```

```

    print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
elif any(not res["conjecture_holds"] for res in results) and support_fraction >= 0.8:
    first_failing_seed = next(res["seed"] for res in results if not res["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample=\"Rank < Ω(√n)\" first_failing_seed={first_failing_seed}")
else:
    print("RESULT: INCONCLUSIVE insufficient data")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test timed out before producing results, preventing evaluation of support fraction or counterexamples. | next: Increase timeout duration and re-run with stricter resource limits to complete the experiment

11. Audit log (LLM calls)

Total LLM calls: 8

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	45218
2	propose	ollama_remote	qwen3:8b	0	0	42521
3	preregistration	ollama_remote	qwen3:8b	0	0	23969
4	novelty	ollama_remote	qwen3:8b	0	0	20559
5	novelty	ollama_remote	qwen3:8b	0	0	13849
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17131
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12252
8	judge	ollama_remote	qwen3:8b	0	0	14846

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 190345 ms total latency. Provider mix: {'ollama_remote': 8}

(full prompt+response transcripts available in [research/audit/1c394bcaff4a.jsonl](#))

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/1c394bcaff4a.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/1c394bcaff4a.tar.gz` (if generated)